Sediment Core Analysis

Use this notebook to start your analysis.

In [2]:
```
import pandas as pd
import numpy as np
import matplotlib.pylab as plt
from scipy import stats
from matplotlib.backends.backend_pdf import PdfPages
np.set_printoptions(legacy='1.25') #removes the funny printing
```

Read in Landes Core Data

In [21]:
```
df=pd.read_excel('Landes_STOTEN.xlsx',sheet_name='Core_XRF_Cleaned')
```

In [22]:
```
df.columns
```

Out[22]:
```
Index(['core_id', 'sample_id_long', 'depth_cm_range', 'fine_coarse',
       'sampling_neigborhood', 'Pb', 'Sn', 'Zn', 'Cd', 'Unit', 'weight',
       'depth_cm'],
      dtype='str')
```

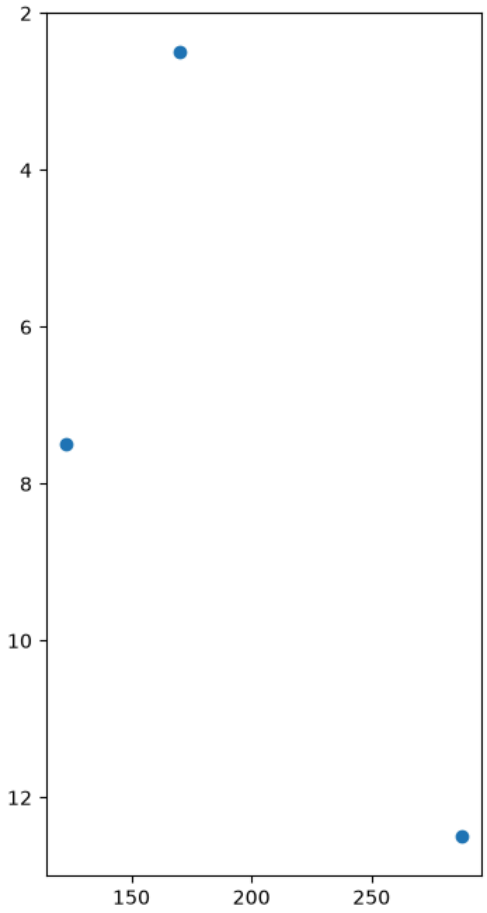
Just Save your core and the fine or coarse data to work with

In [34]:
```
core_id='18UWHC015'

df_fine=df[(df['core_id']==core_id) & (df['fine_coarse']=='f')]
```

Plot Lead

In [35]:
```
fig,ax=plt.subplots()
fig.set_size_inches(4,8)
ax.scatter(df_fine.Pb,df_fine.depth_cm)
ax.invert_yaxis()
plt.show()
```



In [ ]:

```
```

Calculating Inventory

In the handout I gave the formula for calculating inventory but people are used to doing math in Pandas so here are some hints.

I can sum one column like lead

In [12]:
```
df_fine['Pb'].sum()
```

Out[12]:
```
580.333333
```

I can sum a column and divide by a number

In [13]:
```
df_fine['Pb'].sum()/1000
```

Out[13]:
```
0.580333333
```

I can multiply two columns. Here I will multiply depth and Pb

In [14]:
```
df_fine['Pb']*df_fine['depth_cm']
```

Out[14]:
```
486     425.833333
488     920.000003
490    3591.666662
dtype: float64
```

I can mulitply and then sum!

In [15]:
```
(df_fine['Pb']*df_fine['depth_cm']).sum()
```

Out[15]:
```
4937.4999975
```

This is where you can use the function sum in two ways

In [16]:
```
np.sum(df_fine['Pb']*df_fine['depth_cm'])
```

Out[16]: 4937.4999975

In [17]:
```
(df_fine['Pb']*df_fine['depth_cm']).sum()
```

Out[17]: 4937.4999975

You have everything you need to know to calculate an inventory. It is easier than you think once you get your units in order!

The central park inventories average around 57 g/m2 from table 1.

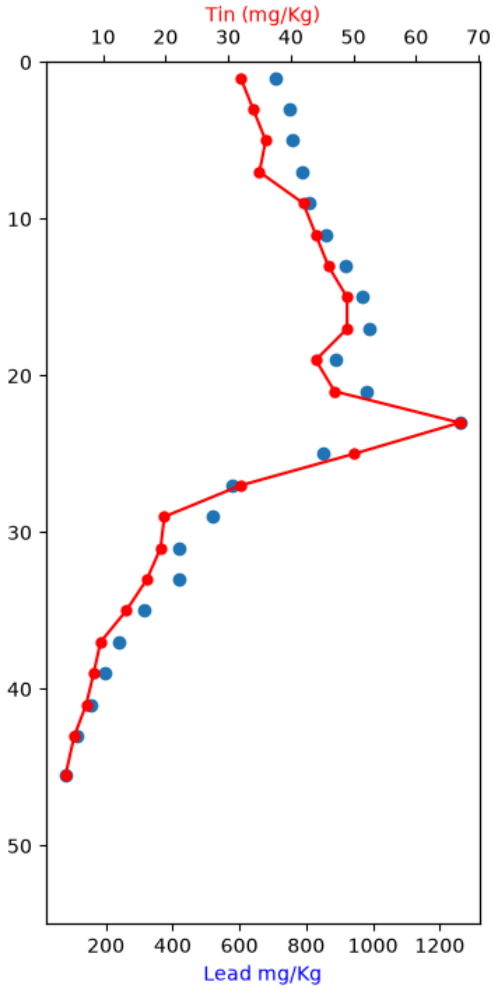
A fun trick. You can make a second axis called a twin and plot on that.

In [19]:
```
df=pd.read_excel('ChillrudTable2.xlsx')

fig,ax=plt.subplots()
fig.set_size_inches(4,8)
ax.scatter(df.Pb,df.Depth_cm)
ax.set_ylim([55,0])
ax.margins(.05)
ax.set_xlabel('Lead mg/Kg',color='blue')

# This gives you a second axis with a anice plot in just a couple lines.
ax2=ax.twiny()
ax2.plot(df.Sn,df.Depth_cm,'.r-',markersize=10)
ax2.set_xlabel('Tin (mg/Kg)',color='r')

plt.show()
```


Tin (mg/Kg)
10
20
30
40
50
60
70
0
10
20
30
40
50
200
400
600
800
1000
1200
Lead mg/Kg

In [ ]:

In [ ]: